K23: Ανάπτυξη Λογισμικού για Πληροφοριακά Συστήματα

Τελική Αναφορά

Σέργιος - Ανέστης Κεφαλίδης
1115201800073

Ελευθερία Λαγκώνη
1115201800088

Χειμερινό εξάμηνο 2021-2022

Περιεχόμενα

1 Εισαγωγή **1**

2 Περιβάλλον υλοποίησης **1**

3 Ο διαγωνισμός **1**

4 Βασική Υλοποίηση **2**

5 Πολυνηματισμός **4**
5.1 Παράλληλη εκτέλεση πολλών MatchDocument 5
5.2 Παράλληλη εκτέλεση μίας MatchDocument 5

6 Αξιολόγηση **6**
6.1 Πληροφορίες 6
6.2 Γενικές παρατηρήσεις από εκτέλεση χωρίς παραλληλία 6
6.3 Rehashing 9
6.4 Διαφορετικές υλοποιήσεις EditDistance 9
6.5 Αναζήτηση σε BK-Tree 10
6.6 Παράλληλη εισαγωγή στοιχείων 11
6.7 Threads 11
6.7.1 Small Dataset 11
6.7.2 30MB Dataset 13
6.7.3 100MB Dataset 14
6.8 Hashtable Implementation 15

1 Εισαγωγή

Η παρούσα αναφορά αναλύει και αξιολογεί τη λύση που αναπτύξαμε για τον διαγωνισμό SIGMOD 2013 στα πλαίσια του μαθήματος "Ανάπτυξη Λογισμικού για Πληροφοριακά Συστήματα".

Ο γενικός κορμός της υλοποίησης περιορίζεται από τα ζητούμενα του μαθήματος. Η χρήση των βασικών δομών ευρετηρίων, συγκεκριμένα Hashtable και BK-Trees ήταν υποχρεωτική. Ταυτοχρόνως, είχαμε την ευχέρεια να αναπτύξουμε δικές μας λύσεις για τις λεπτομέρειες της υλοποίησης καθώς και δικές μας δομές δεδομένων, μιας και η χρήση βιβλιοθηκών περιοριζόταν σε ό,τι περιέχει η libc.

Αρχικά αναλύεται η υλοποίηση σε αλγοριθμικό επίπεδο, τεχνικές λεπτομέρειες μπορούν να αναζητηθούν στο αρχείο README που συνοδεύει τον πηγαίο κώδικα. Στη συνέχεια αναλύεται η εισαγωγή πολυνηματισμού και στο τέλος αξιολογείται η αποτελεσματικότητα της υλοποίησης και διαφόρων παραλλαγών της, είτε αυτές αφορούν την υλοποίηση δομών είτε τις ρυθμίσεις.

2 Περιβάλλον υλοποίησης

- Γλώσσα προγραμματισμού: **C++**
- Λειτουργικά συστήματα: **Ubuntu Linux 18.04, Ubuntu 20.04, PopOS 21.04, PopOS 21.10, Manjaro 21.2.1**
- Build systems: **CMake, Make**
- Testing Framework: **Catch2**

Η C++ επιλέχτηκε ως η γλώσσα υλοποίησης χάρη στον συνδυασμό ταχύτητας και αντικειμενοστραφούς σχεδίασης που προσφέρει. Η ταχύτητα είναι προφανώς απαραίτητη για ένα τέτοιο εγχείρημα. Όσον αφορά την αντικειμενοστραφή φύση της, αυτή έκανε την υλοποίηση ορισμένων τμημάτων, όπως του πολυνηματισμού, εξαιρετικά απλή και άρτια. Παραλλήλως, η δυνατότητα χρήσης πιο "προχωρημένων" εργαλείων όπως οι συναρτήσεις lambda μας επιτρέπουν να διατηρούμε τον κώδικα όσο συντομότερο γίνεται.

3 Ο διαγωνισμός

Ο διαγωνισμός εν συντομία αφορά στη δημιουργία ενός reverse index. Ερωτήματα αποθηκεύονται τα οποία αποτελούνται από λέξεις. Στη συνέχεια έρχονται κείμενα και το πρόγραμμα καλείται να απαντήσει σε ποια ερωτήματα απαντάει το κάθε κείμενο. Ένα κείμενο απαντάει σε κάθε ερώτημα αν περιέχει όλες τις λέξεις του, ή λέξεις που βρίσκονται αρκετά κοντά σύμφωνα με μία συνάρτηση απόστασης (hamming ή levenshtein).

Οι διοργανωτές του διαγωνισμού παρείχαν ένα αρχείο driver το οποίο καλούσε έξι συναρτήσεις μίας βιβλιοθήκης που οι διαγωνιζόμενοι καλούνταν να υλοποιήσουν. Από αυτές τις συναρτήσεις μας ενδιαφέρουν οι τέσσερις, καθώς οι άλλες

δύο ασχολούνται με αρχικοποίηση και αποδέσμευση μνήμης και όποιων δόμων χρησιμοποιούνται.

StartQuery: Αυτή η συνάρτηση εισάγει ένα ερώτημα στο ευρετήριο. Ταυτοχρόνως καθορίζει αν δέχεται μόνο ακριβής συγκρίσεις ή με απόσταση καθώς και το είδος και μέγεθος αυτής της απόστασης.

EndQuery: Δηλώνει ότι ένα ερώτημα δεν είναι πλέον ενεργό.

MatchDocument: Η οποία δίνει ένα κείμενο και το πρόγραμμα καλείται να φυλάξει σε ποια ερωτήματα απαντάει αυτό το κείμενο.

GetNextAvailRes: Η οποία ζητάει να της δοθεί απάντηση για ένα κείμενο. Απάντηση για αυτό το κείμενο δεν πρέπει να έχει δοθεί προηγουμένως.

Σημασία έχει να σχολιάσουμε πως και πότε το πρόγραμμα ζητάει απαντήσεις (GetNextAvailRes). Κάθε φορά που καλείται η MatchDocument το driver πρόγραμμα φυλάει το αποτέλεσμα που περιμένει σε έναν πίνακα. Έτσι μαζεύονται πολλά αναμενόμενα αποτελέσματα. Δεν καλείται όμως η GetNextAvailRes μέχρι να συναντηθεί μία StartQuery ή EndQuery, οπότε και ζητούνται όλα τα αποτελέσματα που έχουν μαζευτεί και στη συνέχεια προχωράει ο driver στη Start/EndQuery.

Από αυτή τη συμπεριφορά καταλαβαίνουμε επίσης ότι μπορούν να καταφθάνουν νέα ερωτήματα σε οποιαδήποτε στιγμή της εκτέλεσης του προγράμματος οπότε το ευρετήριο θα πρέπει να μπορεί να μεταβάλλεται κατά την εκτέλεση.

4 Βασική Υλοποίηση

Από τις απαιτήσεις του μαθήματος ήμασταν υποχρεωμένοι να χρησιμοποιήσουμε ένα πλήθος διαφορετικών ευρετηρίων και δομών. Για τις ερωτήσεις που απαιτούσαν ακριβής αντιστοίχιση (exact matching) έπρεπε να δημιουργηθεί ένα Hashtable ως ευρετήριο. Για τις ερωτήσεις που ζητούσαν edit distance matching ένα BK-Tree του οποίου οι κόμβοι ήταν τύπου Entry. Μία δομή τύπου Entry αποτελείται από ένα αλφαριθμητικό, τη λέξη, και ένα payload, που είναι το πλήθος των ερωτημάτων στα οποία συναντάται η λέξη. Για τα ερωτήματά με hamming distance έπρεπε να δημιουργηθεί ένα πλήθος από BK-Trees, ένα για κάθε πιθανό μέγεθος λέξης. Μιας και η δομή Entry χρησιμοποιείται από τα άλλα 2 βασικά ευρετήρια και για να υπάρχει λογική συνέπεια και η δομή Hashtable χρησιμοποιεί ως δεδομένα δομές Entry. Φυσικά, θα ήταν πιο αποδοτική η χρήση μόνο του payload ως δεδομένο αποθήκευσης, αλλά αυτή είναι μία ασήμαντη επί της παρούσης βελτίωση στη χρήση μνήμης.

Τρία είναι τα βασικά ζητήματα που προβάλλουν από αυτούς τους περιορισμούς. Το πρώτο αφορά τη StartQuery. Πρέπει τα ευρετήρια να μπορούν να μεταβάλλονται καθ όλη την εκτέλεση του προγράμματος.

Αυτό στην περίπτωση ενός hashtable λύνεται τετριμμένα με τη χρήση rehashing. Ένα hashtable μπορεί σε O(1), ή σχεδόν O(1), χρόνο να ξέρει αν περιέχει ένα Entry. Αν ναι τότε μπορεί εύκολα να προσθέσει ένα ακόμα Query στο payload του. Αλλιώς μπορεί να προσθέσει ένα νέο Entry και αν αυτό καθιστά το Hashtable πέρα από το rehashing limit γίνεται rehash.

Στην περίπτωση όμως ενός BK-Tree η αναζήτηση μίας λέξης, αν αυτή υπάρχει, ώστε να προστεθεί στο υπάρχον payload ένα Query δεν είναι τόσο γρήγορη. Για αυτό το λόγο χρειάζεται ένας γρηγορότερος τρόπος για να επεξεργαζόμαστε τα payloads. Η λύση που δώσαμε σε αυτό το πρόβλημα είναι η χρήση Hashtables και κοινών Entries ανάμεσα σε Hashtables και BK-Trees. Με αυτό τον τρόπο διατηρούνται τα θετικά της αναζήτησης σε ένα BK-Tree με τη χρήση μιας συνάρτησης απόστασης, ενώ ταυτοχρόνως η προσθήκη νέων στοιχείων στο payload γίνεται σε χρόνο O(1).

Το άλλο μεγάλο ζήτημα που καλείται η κάθε ομάδα να αντιμετωπίσει είναι η MatchDocument. Αρχικά, σύμφωνα με τις απαιτήσεις του μαθήματος, από κάθε έγγραφο κρατούσαμε μόνο τις μοναδικές λέξεις ώστε να μη χάνεται χρόνος με αλλεπάλληλες αναζητήσεις των ίδιων λέξεων.

Η αναζήτηση για ερωτήματα τύπου exact match ήταν εύκολη μιας και απαιτούνταν μία απλή αναζήτηση στο hashtable index. Όμως, στα υπόλοιπα ευρετήρια, ένα entry μπορεί να περιέχει payload με ερωτήματα που έχουν διαφορετικές απαιτήσεις ελάχιστης απόστασης. Αυτό σημαίνει ότι δεν μπορούμε να θεωρούμε ότι όλες οι queries ενός payload μετράνε μιας και ανάλογα με την παράμετρο απόστασης που μας επέτρεψε να φτάσουμε μέχρι εκείνο το σημείο αυτή η λέξη είναι άκυρη ή έγκυρη για διαφορετικά ερωτήματα. Αυτό σημαίνει ότι για κάθε ερώτημα πρέπει να ελέγχεται αν η παράμετρος απόστασης που δέχεται είναι συμβατή με την παράμετρο απόστασης που χρειάστηκε για να βρεθεί η όμοια λέξη. Αν ναι, τότε αυτή η λέξη μπορεί να θεωρηθεί πώς βρέθηκε. Αλλιώς θα πρέπει να αγνοηθεί και το πρόγραμμα να δράσει σαν εκείνο το Query να μη βρισκόταν στο payload.

Για την αντιμετώπιση αυτού του προβλήματος αναπτύχθηκαν δύο διαφορετικοί αλγόριθμοι αναζήτησης σε BK-Tree που θα αναφερθούν αργότερα και ένα επιπλέον Hashtable που αντιστοιχεί ένα QueryID με ένα Query ώστε να μπορούμε γρήγορα να βρούμε την παράμετρο απόστασης του κάθε ερωτήματος. Επιπλέον, αυτό το Hashtable μας επιτρέπει να ανακτούμε γοργά δεδομένα για τη θέση της κάθε λέξης στο Query που είναι απαραίτητα για τη σωστή λειτουργία της εμπνευσμένης από bitmap δομής που χρησιμοποιούμε για να αναγνωρίζουμε αν ένα ερώτημα έχει απαντηθεί ή όχι.

Το τελευταίο ζήτημα που καλούμαστε να αντιμετωπίσουμε είναι η EndQuery. Επειδή η αφαίρεση στοιχείων από BK-Trees είναι πολύπλοκη, θεωρήσαμε συμφέρον να απενεργοποιούμε τα αντίστοιχα queries.

Μία ελαφρώς απλοποιημένη εκδοχή της MatchDocument:

```
exact_index, edit_index, hamming_index <− indexes created by StartQuery
word_counter_array <− 2d array with dimensions
                          NUMBER_OF_WORDS x (MAXIMUM QUERY LENGTH +
                              ↪ 1)
results = {} <− a list that holds the QueryIDs of matched Queries

foreach unique_word in Document:
    if unique_word exists in exact_index:
        foreach query in unique_words payload:
            word_counter_array[QueryID][5] += 1

    lookup (BK−Tree) word in edit_index for distance 1 to 3 and return 3 separate lists
    foreach list representing one distance:
        foreach word in the list:
            foreach query in its payload:
                if (word_counter_array[QueryID][wordIndex] != 1)
                    word_counter_array[QueryID][5] += 1
                    word_counter_array[QueryID][wordIndex] = 1

    do what you did with edit_index, but this time with hamming_index

for row in range(0, NUMBER_OF_WORDS):
    if word_counter_array[row][5] == number of words in Query represented by row:
        results.append(row/QueryID)
```

5 Πολυνηματισμός

Σημαντικό ρόλο στην προσπάθεια μείωσης του χρόνου εκτέλεσης διαδραματίζει η εισαγωγή παραλληλίας στην εκτέλεση. Η παραλληλία ως έννοια εφαρμόζεται σε πολλά διαφορετικά επίπεδα ενός υπολογιστικού συστήματος, από τον επεξεργαστή μέχρι τη χρήση νημάτων ή πολλών διεργασιών. Στη δική μας περίπτωση η παραλληλία υλοποιήθηκε μέσω της χρήσης νημάτων.

Το ιδανικό αποτέλεσμα της εισαγωγής παραλληλίας επιπέδου νημάτων σε ένα πρόγραμμα είναι η διαίρεση του χρόνου εκτέλεσης με το πλήθος των πυρήνων του συστήματος (ή των νημάτων αν αυτά είναι λιγότερα από τους ελεύθερους πυρήνες). Αυτό φυσικά σπάνια καθίσταται δυνατόν, μιας και ο συγχρονισμός και η προστασία των δεδομένων εισάγουν κάποιες επιπλέον καθυστερήσεις στα περισσότερα είδη προγραμμάτων.

Στην περίπτωση του προγράμματος του διαγωνισμού η πιο εμφανής και χρήσιμη μορφή παραλληλίας είναι στη συνάρτηση MatchDocument. Το πρόγραμμα χρησιμοποιεί τη συντριπτική πλειοψηφία των κύκλων ρολογιού που του δίνονται σε αυτή τη συνάρτηση, που αποτελεί άλλωστε και το "ζουμί" της υλοποίησης. Ένα άλλο πιθανό σημείο παραλληλίας είναι στην εισαγωγή ερωτημάτων στα ευρετήρια. Οι κλήσεις στη StartQuery είναι πάρα πολλές, όπως μπορεί να δει κάποιος στα αρχεία δεδομένων του διαγωνισμού.

Ένα σημαντικό ερώτημα, σε αυτό το σημείο, είναι αν είναι προτιμότερη η παράλληλη επεξεργασία πολλών MatchDocument ή η παραλληλοποίηση της ίδιας της MatchDocument. Υλοποιήσαμε και τις δύο τεχνικές, όπως θα παρουσιαστούν στη συνέχεια.

5.1 Παράλληλη εκτέλεση πολλών MatchDocument

Η παράλληλη εκτέλεση πολλών MatchDocument έχει, θεωρητικά, το πλεονέκτημα της μικρότερης αδράνειας πυρήνων. Από τη στιγμή που κάθε νήμα καλείται να απαντήσει για ένα διαφορετικό Document, έχει και δικό του αντίγραφο της δομής που αποθηκεύει τα αποτελέσματα της αναζήτησης δεν χρειάζεται να περιμένει για την επεξεργασία αυτής της δομής. Έτσι, κάθε πυρήνας μπορεί να τρέχει στο 100% των δυνατοτήτων. Η μεγαλύτερη αδυναμία αυτής της προσέγγισης είναι η διαφορά του απαιτούμενου χρόνου επεξεργασίας κάθε Document. Ορισμένα Documents χρειάζονται πολλαπλάσιο χρόνο από άλλα μέχρι να ολοκληρωθεί η διαδικασία αναζήτησης, με αποτέλεσμα αν ένα τέτοιο Document είναι από τα τελευταία μίας ομάδας να εκτελείται μόνο ένα νήμα ενώ τα υπόλοιπα το περιμένουν.

Ταυτοχρόνως, λόγω της ύπαρξης πολλών δομών φύλαξης πληροφοριών αναζήτησης, που όπως αναφέρθηκε στη δική μας περίπτωση μοιάζει με bitmap, η χρήση μνήμης είναι αυξημένη. Αυτό θα μπορούσε να αποτελέσει πρόβλημα σε ένα σύστημα με πολλούς πυρήνες, ή σε μία εκτέλεση με πολύ μεγάλο πλήθος νημάτων.

Επιπροσθέτως, αν το πρόγραμμα οδήγησης δεν ήταν αρκετά έξυπνο ώστε να μπορεί να δεχτεί απαντήσεις για Documents εκτός σειράς (προσοχή: όχι ασύγχρονα) θα έπρεπε να υπάρχει ένας μηχανισμός σωστής τοποθέτησης των αποτελεσμάτων των MatchDocument ώστε να δίνονται με τη σωστή σειρά. Ευτυχώς, κάτι τέτοιο δεν χρειάζεται στη συγκεκριμένη περίπτωση.

5.2 Παράλληλη εκτέλεση μίας MatchDocument

Υπάρχουν πολλοί πιθανοί τρόποι για την παραλληλία σε επίπεδο MatchDocument. Μία ιδέα είναι η ταυτόχρονη αναζήτηση σε διαφορετικά ευρετήρια. Άλλη δυνατότητα είναι η εισαγωγή παραλληλίας στη δομή του BK-Tree, στην οποία διατίθεται και η συντριπτική πλειοψηφία του επεξεργαστικού χρόνου. Εμείς αποφασίσαμε ότι ο πιο συμφέρον και ταυτόχρονα εύκολος τρόπος υλοποίησης είναι η δημιουργία μίας δουλειάς (job) για κάθε λέξη του Document και η παράλληλη αναζήτηση διαφορετικών λέξεων.

Αυτή η λύση είναι και εύκολη στην υλοποίηση και θα οδηγεί σε σχετικά μικρή αδράνεια πυρήνα σε συστήματα όπως αυτό που είχαμε διαθέσιμο για ανάπτυξη και αξιολόγηση (περισσότερα στη συνέχεια). Σε ένα σύστημα με πολύ μεγάλο πλήθος πυρήνων, αυτή η λύση θα είχε πολύ χειρότερα αποτελέσματα, μιας και οι πιθανότητες πολλοί πυρήνες να περιμένουν για να επεξεργαστούν την ίδια δομή (bitmap) θα ήταν μεγάλη. Αυτό το ζήτημα θα μπορούσε ίσως να λυθεί με την χρήση περισσότερων δομών και στο τέλος την πρόσθεση των στοιχείων τους, αλλά αυτό εισάγει αλγοριθμική, και κατ' επέκταση επεξεργαστική πολυπλοκότητα. Επίσης, μία τέτοια λύση θα αύξανε τη χρήση μνήμης.

Ένα άλλο σημείο ενδιαφέροντος είναι ότι μιας και οι "δουλειές" είναι πολύ μικρότερες, σε ένα σύστημα με περισσότερους πυρήνες θα ήταν δύσκολο να τροφοδοτεί το κύριο νήμα εκτέλεσης όλα τα νήματα αναζήτησης.

6 Αξιολόγηση

6.1 Πληροφορίες

Κάθε αποτέλεσμα προέκυψε ως ο μέσος όρος τριών διαδοχικών εκτελέσεων. Οι προδιαγραφές του συστήματος που χρησιμοποιήθηκε είναι οι ακόλουθες:

- CPU: Ryzen 2400g (4c/8t) @ 3.6Ghz
- RAM: Dual Channel DDR4 @ 3000 Mhz
- SSD: Samsung EVO 850
- OS: Manjaro Linux 21.2.1
- Kernel: 5.10.89
- Compiler: GCC 11.1.0 με βελτιστοποιήσεις -O3
- Desktop Environment: Cinnamon (1% CPU usage)

Για την εκτέλεση του προγράμματος και την αξιολόγηση της αποτελεσματικότητας του πολυνηματισμού απενεργοποιήθηκε η λειτουργία Clock Boosting που ανεβάζει τον ρυθμό ρολογιού στα 3.9Ghz κατά την εκτέλεση σε έναν πυρήνα. Αυτό μας επιτρέπει να αξιολογούμε την αποτελεσματικότητα του αλγορίθμου που αναπτύξαμε και όχι του επεξεργαστή. Επίσης, αφαιρεί την πιθανότητα το πρόγραμμα να βγάζει διαφορετικά αποτελέσματα ανάλογα με τη θερμοκρασία του επεξεργαστή εκείνη τη στιγμή (π.χ. ύστερα από διαδοχικές εκτελέσεις).

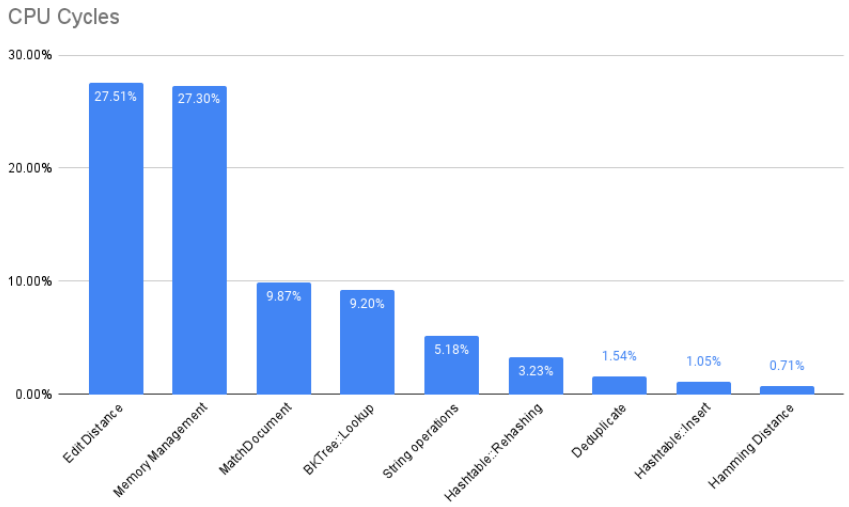
Αυτό που χρησιμοποιήθηκε είναι η τεχνολογία Simultaneous multithreading η οποία επιτρέπει την παράλληλη εκτέλεση 2 νημάτων σε έναν φυσικό πυρήνα. Αυτό μας επιτρέπει να εξερευνήσουμε την απόδοση σε συστήματα με μεγαλύτερο πλήθος πυρήνων, αλλά φυσικά αυτοί οι εικονικοί πυρήνες δεν έχουν την ίδια απόδοση, κάτι που θα φανεί και στη συνέχεια.

6.2 Γενικές παρατηρήσεις από εκτέλεση χωρίς παραλληλία

Συνολικά έχουμε στη διάθεση μας τέσσερα αρχεία με δεδομένα εισόδου για το πρόγραμμα. Ένα απο αυτά, με όνομα small_test, είναι αρκετά μικρό ώστε να εκτελείται σε λιγότερο από ένα λεπτό ακόμα και από την εξαιρετικά βασική υλοποίηση των διοργανωτών του SIGMOD. Τα υπόλοιπα τρία, μεγέθους 30, 100 και 1000 MB αντίστοιχα, αποτελούν μία εντελώς διαφορετική κατηγορία όσον αφορά τις απαιτήσεις τους. Εξαιρετικά ευνοϊκό είναι ότι οι παρατηρήσεις μας στα αρχείο των 30MB γενικεύουν πολύ καλά στα μεγαλύτερα αρχεία, πράγμα που κάνει τη διενέργεια πολλών δοκιμών ευκολότερη.

Σε αυτή την ενότητα θα αναφερθούμε σε γενικές παρατηρήσεις που κάναμε όσον αφορά τη συμπεριφορά της υλοποίησης μας, με τη βοήθεια του εργαλείου perf και χωρίς παραλληλία. Σε αυτό το σημείο ο στόχος είναι η κατανόηση της συμπεριφοράς των διαφόρων τμημάτων της υλοποίησης.

Αυτό μας φέρνει στο πρώτο διάγραμμα που απεικονίζει την κατανομή κύκλων ρολογιού σε συναρτήσεις. Θα παρατηρήσετε ότι το σύνολο των ποσοστών δεν φτάνει το 100% μιας και ένα 15% καταναλώνεται από όλο το υπόλοιπο πρόγραμμα.



Αξίζει να σημειωθεί ότι το perf μπορεί να βγάζει ελαφρώς διαφορετικά αποτελέσματα και αναλογίες από εκτέλεση σε εκτέλεση, αλλά η γενική εικόνα παραμένει ίδια.

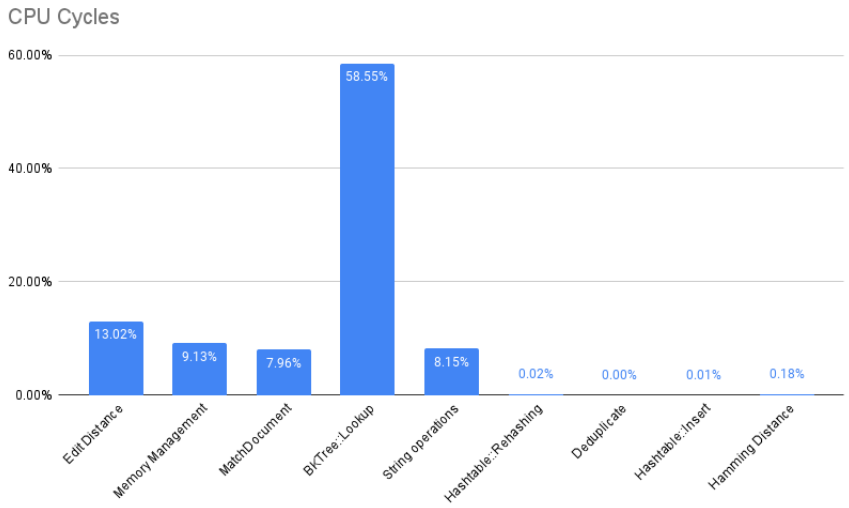
Αρχικά, παρατηρούμε ότι η διαχείριση μνήμης (malloc, free, new, delete) καταναλώνει ένα σημαντικό κομμάτι των κύκλων ρολογιού. Το πρόγραμμα σίγουρα δεσμεύει και απελευθερώνει μεγάλα κομμάτια μνήμης. Μία πιο προχωρημένη υλοποίηση θα μπορούσε να φροντίζει να θυμάται περισσότερα δεδομένα ώστε να μειώνονται τα νέα allocations και οι επαναλαμβανόμενοι υπολογισμοί. Σε αυτή την κατηγορία συμπεριλαμβάνονται διάφορες συναρτήσεις που το perf δεν μπορεί να παρακολουθήσει με ακρίβεια μιας και είναι συναρτήσεις συστήματος.

Όσον αφορά τα στοιχεία του προγράμματος που έχουμε πλήρη εικόνα, φαίνεται καθαρά πόσο σημαντική και βαριά είναι η editDistance, και αυτό σε ένα μικρό αρχείο όπου τα BKTrees είναι σχετικά μικρά. Ταυτοχρόνως, γνωρίζουμε ότι η Edit Distance καλείται λιγότερες φορές κατά την είσοδο ερωτημάτων στο ευρετήριο σε σχέση με την αναζήτηση όπου καλείται πολλαπλάσιες φορές, κάτι που είναι εμφανές από τον αλγόριθμο αναζήτησης. Σε συνδυασμό με την BKTree:Lookup που παίρνει από μόνη της περισσότερο από 9% των κύκλων βλέπουμε ότι η αναζήτηση στο edit index κατά πάσα πιθανότητα καταλαμβάνει πάνω από το ένα τρίτο του χρόνου εκτέλεσης.

Ένα άλλο αξιόλογο γεγονός είναι το κόστος του rehashing. Σε συνδυασμό με την αποδέσμευση και δέσμευση μνήμης που έχει συμπεριληφθεί στην κατηγορία του memory management, προκύπτει ένα μεγάλο κόστος για ένα μικρό σύνολο δεδομένων. Μία πιθανή βελτίωση σε αυτό το μικρό dataset είναι η απενεργοποίηση του rehashing.

Η συμπεριφορά του προγράμματος αλλάζει με μεγαλύτερα σύνολα δεδομένων,

όπως φαίνεται στο ακόλουθο διάγραμμα.



Εδώ φαίνεται ξεκάθαρα πως κυριαρχεί το κόστος αναζήτησης σε BKTree, και συγκεκριμένα στο edit_index. Πλέον το κόστος χρήσης Hashtable και της συνάρτησης Deduplicate είναι αμελητέο. Η αναζήτηση στα BKTrees με hamming distance είναι πολύ λιγότερο κοστοβόρα, αν παρατηρήσουμε ότι το κόστος της Hamming Distance είναι κατά 2 τάξης μεγέθους μικρότερος από αυτό της Edit Distance.

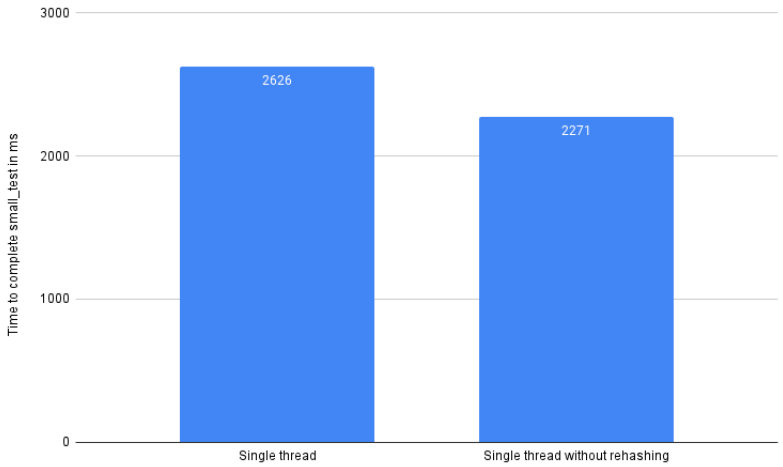
Επιπλέον, το κόστος δέσμευσης και αποδέσμευσης μνήμης πέφτει σημαντικά σε σχέση με τα υπόλοιπα στοιχεία, πράγμα λογικό αφού το πλήθος των Queries στο σύστημα αυξάνεται σχετικά λίγο στα μεγαλύτερα αρχεία, κυρίως γίνονται πολύ περισσότερες κλήσεις στη MatchDocument.

Αυτή η αλλαγή συμπεριφοράς, δικαιολογεί και γιατί μπορούμε να γενικεύσουμε τα αποτελέσματα μας στο αρχείο των 30mb στα μεγαλύτερα αρχεία, αλλά όχι αυτά του small dataset στα μεγαλύτερα. Το small αρχείο, είναι τόσο μικρό ώστε να περνάει μεγάλο κομμάτι του χρόνου του στην κατασκευή και διάλυση των υποδομών της υλοποίησης, σε αντίθεση με τα μεγαλύτερα αρχεία που χρησιμοποιούν το μεγαλύτερο κομμάτι του χρόνου τους για παραγωγή αποτελεσμάτων.

Αυτό όμως που μπορούμε να κάνουμε, είναι να δοκιμάσουμε βελτιώσεις στην lookup στο μικρό dataset και, ανάλογα με το στόχο, να κρίνουμε αν μία βελτίωση θα φέρει αρνητικά ή θετικά αποτελέσματα σε μεγαλύτερα αρχεία. Αυτό γίνεται διότι στις σχετικά απλές (σε σχέση με τις υλοποιήσεις των κορυφαίων ομάδων του διαγωνισμού) βελτιστοποιήσεις που δοκιμάσαμε δεν υπάρχει στάδιο υποδομής το οποίο θα μπορούσε να δίνει αρνητικά αποτελέσματα για μικρά datasets και θετικά για μεγάλα.

Αυτή η θεωρία, επιβεβαιώνεται και στα πειράματα που ακολουθούν.

6.3 Rehashing



Όπως φαίνεται στο γράφημα, η απενεργοποίηση του rehashing δίνει μία σημαντική βελτίωση στον χρόνο εκτέλεσης. Αυτό γίνεται επειδή οποιοδήποτε κέρδος από το rehashing είναι πολύ μικρό μπροστά στο χρόνο που χρειάζεται για να αποδεσμευτεί και δεσμευθεί νέα μνήμη και να επανενταχθούν όλα τα στοιχεία.

Η ίδια σύγκριση για το αρχείο των 30mb δείχνει τη σημασία του rehashing σε μεγαλύτερα σύνολα δεδομένων. Η εκτέλεση που το έχει ενεργοποιημένο χρειάζεται περίπου 34 λεπτά, ενώ χωρίς δεν τερμάτισε ούτε ύστερα από δύο ώρες οπότε και την σταματήσαμε.

Ένα άλλο ενδιαφέρον στοιχείο της σύγκρισης στο αρχείο των 30mb είναι η χρήση μνήμης. Το rehashing οδηγεί στη δέσμευση περίπου 600 επιπλέον megabyte μνήμης (από τα 1900 στα 2500). Φαίνεται, βάση αυτού, πόσο συμφέρουσα είναι η χρήση του rehashing αφού η αύξηση της χρήσης της μνήμης κατά το ένα τρίτο μειώνει πάρα πολύ τον χρόνο εκτέλεσης.

6.4 Διαφορετικές υλοποιήσεις EditDistance

Βλέποντας το τεράστιο κόστος της edit distance, προσπαθήσαμε να βελτιώσουμε τη συνάρτηση editDistance μας. Οι βασικές βελτιώσεις, αρχικά χρήση πίνακα για δυναμικό προγραμματισμό, και στη συνέχεια μία γραμμής και 2 στοιχείων δίνουν τα αποτελέσματα που παρουσιάστηκαν παραπάνω.

Μία περαιτέρω βελτίωση που εξερευνήθηκε είναι η σύγκριση του προθέματος και της κατάληξης των λέξεων, με την ελπίδα ότι αυτές θα είναι αρκετά όμοιες ώστε να συγκρίνονται λιγότερα, κεντρικά γράμματα. Δυστυχώς, αυτή η προσπάθεια αποδείχθηκε άκαρπη. Οι λέξεις με κοινά στοιχεία στην αρχή και στο τέλος είναι τόσο λίγες ώστε, τελικά, να αυξάνεται ελάχιστα ο χρόνος εκτέλεσης αντί να μειώνεται.

6.5 Αναζήτηση σε BK-Tree

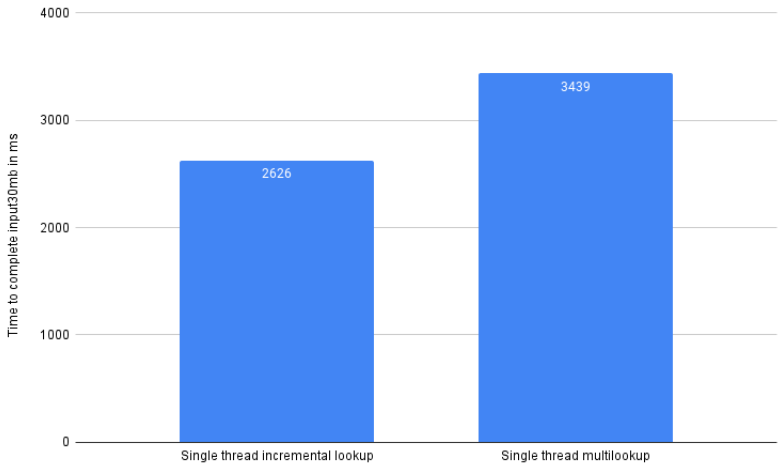
Συνεχίζοντας τη σκέψη μείωσης του χρόνου εκτέλεσης της αναζήτησης στο edit_index που όπως δείξαμε προηγουμένως αποτελεί τη βασική πηγή καθυστέρησης, και όπως θα δειχθεί στη συνέχεια ακόμα μεγαλύτερη σε μεγάλα αρχεία, θα αναλυθούν πιθανές βελτιώσεις του αλγορίθμου lookup.

Μίας και κάθε Entry node περιέχει ένα payload με λέξεις από ερωτήματα με διαφορετική ανεκτή απόσταση, υπάρχει η ανάγκη να αναζητούνται στο BKTree οι λέξεις για όλες τις δυνατές αποστάσεις (1, 2, 3) και ανάλογα με την απόσταση που χρειάστηκε για να φτάσουμε σε κάθε κόμβο να αντιμετωπίζουμε ανάλογα το payload.

Ο απλούστερος τρόπος για την επίλυση αυτού του ζητήματος είναι η κλήση της κλασικής BKTree::lookup τρεις φορές. Αυτός ο αλγόριθμος αναφέρεται σε μεγάλο πλήθος διαδικτυακών πηγών και οπότε δεν θεωρούμε σκόπιμο να τον αναλύσουμε. Το βασικό πρόβλημα με αυτή την προσέγγιση είναι η πολλαπλή επεξεργασία των ίδιων στοιχείων. Η δεύτερη αναζήτηση θα περάσει από τους ίδιους κόμβους που θα περάσει και η πρώτη, συν όσους κόμβους δεν περνάει η πρώτη. Αντιστοίχως για την τρίτη επανάληψη. Αυτό θυμίζει τον αλγόριθμο iterative-dfs που εφαρμόζεται σε προβλήματα αναζήτησης ή/και κλασικής τεχνητής νοημοσύνης. Αυτό μας επιτρέπει να κάνουμε μία πρόβλεψη, ότι το επιπλέον αυτό κόστος δεν θα είναι μεγάλο σε σχέση με το κόστος της τελευταίας επανάληψης και οπότε δεν θα επηρεάζει πάρα πολύ το τελικό αποτέλεσμα.

Μία άλλη λύση είναι η ταυτόχρονη εξερεύνηση και των τριών πιθανών αποστάσεων. Η λογική αυτής της λύσης είναι αντί για το διάστημα [distance - maxDistance, distance + maxDistance], να εξερευνώνται τα διαστήματα $[distance - 1, distance+1], ..., [distance-n, distance-n+1] \cup [distance+n-1, distance+n]$. Κατά την εκτέλεση διατηρούνται τρεις διαφορετικές λίστες με αποτελέσματα και για κάθε στοιχείο που μπαίνει στην ουρά των υποψήφιων φυλάσσεται επίσης το ελάχιστο κόστος για την εξερεύνηση του στοιχείου. Με άλλα λόγια, για κάθε υποψήφιο στοιχείο γνωρίζουμε επίσης τη μικρότερη μέγιστη απόσταση που θα χρειαζόταν για να το εξερευνήσει. Στο τέλος επιστρέφονται οι τρεις λίστες, κατανέμοντας κατάλληλα τα στοιχεία σύμφωνα με αυτή την πληροφορία απόστασης.

Παρόλο που αυτός ο αλγόριθμος ακούγεται σαν μία καλή ιδέα, το κόστος των εντολών ελέγχου ροής, και κατά πάσα πιθανότητα αστοχίες κρυφής μνήμης, οδηγούν σε αρνητικά αποτελέσματα. Ο χρόνος εκτέλεσης ανεβαίνει αισθητά για το small_test, ενώ για τα μεγαλύτερα αρχεία γίνεται απαγορευτική, χρονικά, η χρήση αυτού του αλγορίθμου.



6.6 Παράλληλη εισαγωγή στοιχείων

Από την αρχική μας ανάλυση της συμπεριφοράς του προγράμματος είδαμε ότι η είσοδος στοιχείων στα ευρετήρια χρειάζεται αμελητέο χρόνο σε σχέση με την αναζήτηση. Παρόλα αυτά, υλοποιήσαμε τις απαραίτητες δομές συγχρονισμού για την παράλληλη εισαγωγή στοιχείων. Τα αποτελέσματα ήταν τα αναμενόμενα, με τις διαφορές να είναι στατιστικά ασήμαντες (παρόμοιες με τις διαφορές από εκτέλεση σε εκτέλεση) οπότε αυτή η βελτιστοποίηση δεν υλοποιήθηκε και στην παραλληλία επιπέδου λέξης.

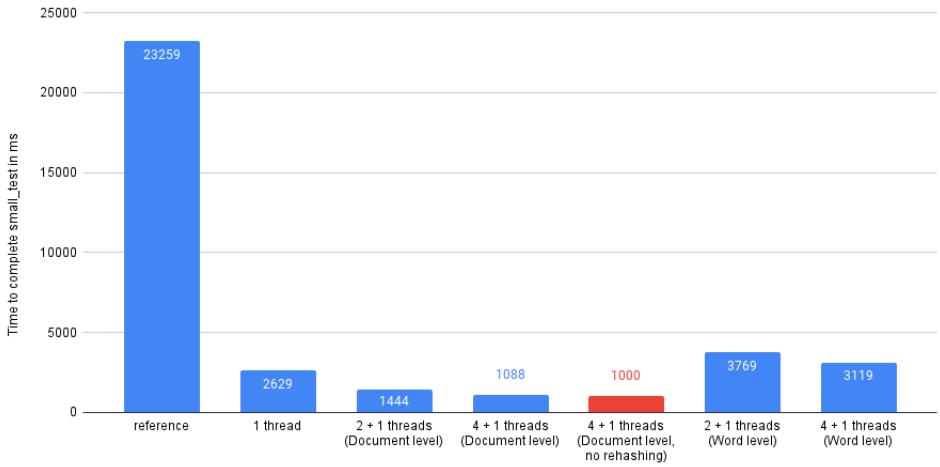
6.7 Threads

Σε αυτή την ενότητα θα αναλυθεί η αποδοτικότητα των διαφορετικών μεθόδων πολυνηματισμού που αναφέρθηκαν προηγουμένως. Υπενθυμίζεται ότι η μία μέθοδος είναι η παράλληλη αναζήτηση σε διαφορετικά Documents και η άλλη η παράλληλη αναζήτηση λέξεων του ίδιου Document.

Σημειώνεται επίσης ότι η υλοποίηση μας χρησιμοποιεί ένα νήμα για να κάνει το Job Scheduling. Αυτό σημαίνει ότι όταν αναφερόμαστε σε τέσσερα νήματα εργασίας, εννοούμε $4 + 1$ νήματα. Το ένα, κεντρικό νήμα δεν συνεισφέρει ιδιαίτερα στην ταχύτητα του προγράμματος, αλλά βοηθάει στην ανάπτυξη καθαρού και κατανοητού κώδικα.

6.7.1 Small Dataset

Όλες οι εκτελέσεις εκτός από την ταχύτερη έγιναν με rehashing ώστε τα δεδομένα να μπορούν να συγκριθούν με τις επόμενες ενότητες. Επίσης, συμπεριλαμβάνεται η υλοποίηση - παράδειγμα του διαγωνισμού ως σημείο αναφοράς της δυναμικής του συστήματος που χρησιμοποιήθηκε.

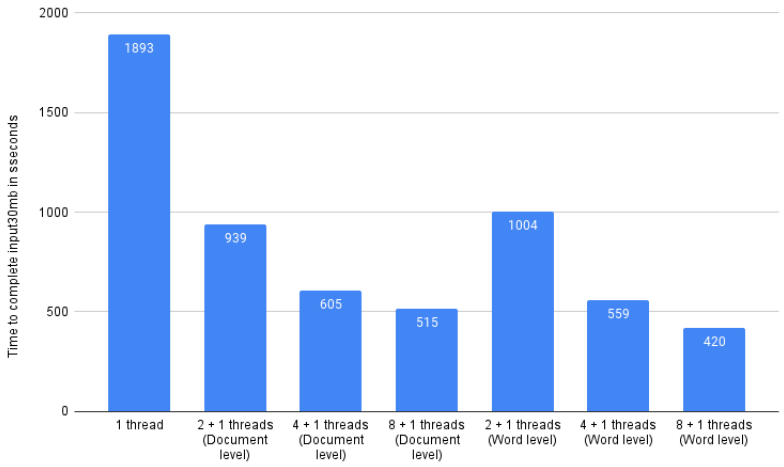


Παρατηρούμε αρχικά ότι η υλοποίηση μας είναι σημαντικά ταχύτερη από άποψη αλγορίθμου, πράγμα αναμενόμενο μιας και η υλοποίηση του sigmod χρησιμοποιεί απλοϊκές δομές για το πρόβλημα. Το κύριο σημείο ενδιαφέροντος είναι η σχέση ανάμεσα στις διαφορετικές υλοποιήσεις πολυνηματισμού.

Στην υλοποίηση επιπέδου λέξης το overhead της υλοποίησης είναι τόσο μεγάλο ώστε να αυξάνεται ο χρόνος εκτέλεσης. Ταυτοχρόνως, αυτός ο αλγόριθμος χρειάζεται κάποιο χρόνο μέχρι να τροφοδοτήσει όλα τα νήματα, και στη συγκεκριμένη περίπτωση αυτά τελειώνουν τόσο γρήγορα τη δουλειά τους ώστε να περιμένουν το κύριο νήμα να εισάγει νέες δουλειές.

Η υλοποίηση σε επίπεδο document, από την άλλη, δεν αντιμετωπίζει προβλήματα τροφοδοσίας και η deduplicate τρέχει σε κάθε ξεχωριστό νήμα. Αποτέλεσμα είναι στην περίπτωση των 2 νημάτων η επιτάχυνση να είναι σχεδόν ιδεατή και στα 4 νήματα να πλησιάζει τριψήφιο αριθμό χιλιοστών του δευτερολέπτου. Επιβεβαιώνεται επίσης, ότι είναι προτιμότερη η απενεργοποίηση του rehashing σε ένα τόσο μικρό σύνολο δεδομένων.

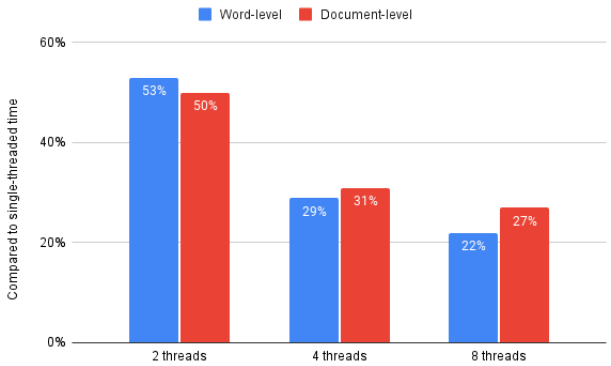
6.7.2 30MB Dataset



Περνάμε τώρα σε ένα πολύ μεγαλύτερο και πιο αντιπροσωπευτικό του στόχου του διαγωνισμού σύνολο δεδομένων. Το σύνολο δεδομένων των 30mb αποτελείται από χιλιάδες Queries και 60 περίπου Documents, κάποια από τα οποία είναι σημαντικά πιο πολύπλοκα (με μεγαλύτερο πλήθος λέξεων) από αυτά του προηγούμενου dataset. Αυτό είναι και το σύνολο δεδομένων με τις πιο εκτενής δοκιμές, μιας και οι παρατηρήσεις μας γενικεύονται αρκετά καλά και στα μεγαλύτερα.

Συγκρίνοντας τις δύο υλοποιήσεις πολυνηματισμού παρατηρούμε ότι ενώ αρχικά, στην περίπτωση των δύο worker threads, η παραλληλία σε επίπεδο document δίνει καλά αποτελέσματα, οι διαδοχικές αυξήσεις νημάτων δίνουν συγκριτικά μικρότερες βελτιώσεις.

Αντιθέτως, η υλοποίηση σε επίπεδο λέξης, δίνει σχεδόν ιδανικά αποτελέσματα σε κάθε αύξηση νημάτων (υπενθυμίζουμε ότι η τεχνολογία SMT αναμένεται να δίνει περίπου 20% επιπλέον επεξεργαστικής δύναμης σε εφαρμογές που είναι φιλικές προς αυτή). Αυτά φαίνονται και στο παρακάτω διάγραμμα:



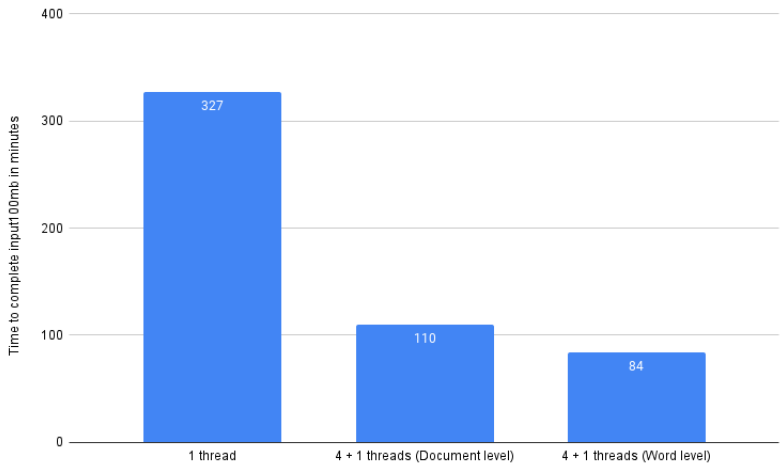
Από αυτά, καταλήγουμε ότι η τεχνική παραλληλίας σε επίπεδο λέξης αντιμετω-

πίζει καλύτερα μεγαλύτερο αριθμό νημάτων, σε αντίθεση με την τεχνική παραλληλίας σε επίπεδο document που δε φαίνεται κατάλληλη για συστήματα με περισσότερους πυρήνες. Αυτό είναι αναμενόμενο, όσο περισσότεροι είναι οι πυρήνες/νήματα τόσο περισσότεροι επεξεργαστική πόροι παραμένουν σε αδράνεια όταν περιμένουν την ολοκλήρωση της αναζήτησης για το τελευταίο document. Ο επεξεργαστής χρησιμοποιείται στο μέγιστο, μέχρι που αρχίζουν ορισμένοι πυρήνες να μην έχουν δουλειά και έτσι διαδοχικά καταλήγουν να περιμένουν έναν.

Αντιθέτως, στο επίπεδο λέξης όσο μπορεί το κύριο νήμα να τροφοδοτεί τα επιμέρους νήματα με λέξεις αυτά δουλεύουν. Ενδιαφέρον έχει ότι το συνολικό CPU utilization είναι ελάχιστα χαμηλότερο σε αυτή την περίπτωση, λόγω της ανάγκης συγχρονισμού και της αδυναμίας του κύριου νήματος να τροφοδοτεί όλα τα νήματα ορισμένες στιγμές (όταν είναι πολύ εύκολες οι δουλειές που τους αναθέτει).

Ένα απροσδόκητο αποτέλεσμα είναι η καλύτερη, πρακτικά ιδανική, απόδοση της τεχνικής επιπέδου document στην περίπτωση των 2 νημάτων. Μιας και ακόμα σε αυτή την περίπτωση παρατηρήθηκε η αδράνεια ενός πυρήνα ενώ ο άλλος δούλευε θεωρούμε ότι αυτό το αποτέλεσμα ήταν μάλλον αποτέλεσμα τύχης (η μία από τις τρεις εκτελέσεις ήταν σημαντικά χαμηλότερη χρονικά, ίσως λόγω λιγότερης χρήσης CPU στο παρασκήνιο από το λειτουργικό σύστημα).

6.7.3 100MB Dataset



Στο μεγαλύτερο σύνολο δεδομένων εισόδου που δοκιμάσαμε, βλέπουμε δύο πράγματα. Αρχικά διατηρείται η παραλληλία σε επίπεδο λέξης ως συμφέρουσα σε σχέση με την παραλληλία σε επίπεδο document.

Ενδιαφέρον έχει, ότι η ψαλίδα της διαφοράς ανοίγει. Αυτό δείχνει ότι όσο μεγαλώνει το index και τα ερωτήματα κάνουν ακόμα περισσότερο χρόνο για να απαντηθούν, τόσο περισσότερο χρόνο περιμένουν τον τελευταίο πυρήνα του batch οι υπόλοιποι πυρήνες.

Με τη χρήση της word-level παραλληλίας φτάνουμε στο 25,68% του χρόνου

εκτέλεσης του ενός νήματος. Αυτό είναι απρόσμενα κοντά στο ιδεατό (μετρώντας τους πυρήνες εργασίας) και φανταζόμαστε ότι είναι αποτέλεσμα λίγων ταυτόχρονων κλειδωμάτων σε μεγαλύτερα αρχεία. Με άλλα λόγια τα νήματα εργασίας συγχρονίζονται έτσι ώστε να μην περιμένουν συχνά για πρόσβαση στη wordCounterArray. Το κεντρικό νήμα δεν μπορεί να ευθύνεται για αυτό το αποτέλεσμα, αφού δεν προσφέρει κάτι από άποψη παραλληλίας μιας και τη μόνη "βαριά" διεργασία δεν την κάνει ασύγχρονα (deduplicate).

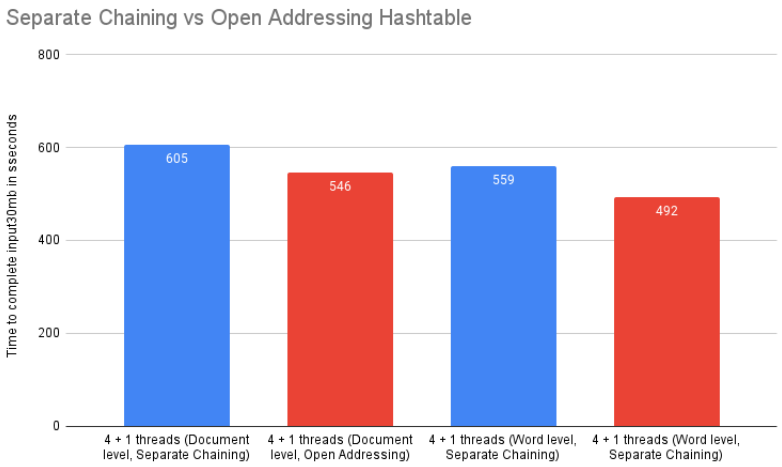
Θεωρούμε ότι αυτά τα αποτελέσματα θα είναι πολύ κοντά και στα αποτελέσματα του αρχείου του ενός GB αλλά μιας και η εκτέλεση τους 3 φορές με διαφορετικές ρυθμίσεις θα χρειαζόταν εξαιρετικά πολύ χρόνο, δεν έχουμε δεδομένα για αυτό.

6.8 Hashtable Implementation

Η τελευταία δοκιμή που έγινε, και για αυτό δεν παρουσιάζονται συνδυαστικά με τις υπόλοιπες αποτελέσματα, είναι μία διαφορετική υλοποίηση του Hashtable. Θεωρούμε γνωστές και τις δύο αυτές βασικές μορφές hashtable. Η χρήση open addressing αντί separate chaining δοκιμάστηκε στο small dataset και σε αυτό των 30mb.

Στο μικρό dataset τα αποτελέσματα ήταν αναμενόμενα. Δεν υπήρξε κάποια μετρήσιμη διαφορά σε χρόνο εκτέλεσης η χρήση μνήμης. Αντιθέτως, στο dataset των 30mb παρατηρήθηκε μεγάλο κέρδος, ειδικά όσον αφορά την κατανάλωση μνήμης. Από τα 2.5GB η κατανάλωση μνήμης του προγράμματος έπεσε στα 1.6GB, μειώθηκε δηλαδή στα 2/3.

Επιπλέον, στην εκτέλεση των τεσσάρων (συν ένα) νημάτων παρατηρήθηκε μία αρκετά μεγάλη βελτίωση της ταχύτητας ολοκλήρωσης του προγράμματος, όπως φαίνεται και στο παρακάτω διάγραμμα:



Και στους δύο αλγορίθμους πολυνηματισμού βλέπουμε μία μείωση του χρόνου εκτέλεσης κοντά στο 10%. Ο πίνακας κατακερματισμού είναι η κυρίαρχη δομή που χρησιμοποιούμε (πέρα από τα ευρετήρια) και έτσι εξηγείται και η θεαματική

μείωση στη χρήση μνήμης (πολύ λιγότερες λίστες) αλλά και η σημαντική μείωση στο χρόνο, αφού από ότι φαίνεται οδηγεί σε λιγότερες συγκρίσεις αλφαριθμητικών. Σημειώνεται σε αυτό το σημείο ότι η υλοποίηση μας για separate chaining έχει περιθώρια βελτίωσης, όσον αφορά τη χρήση μνήμης καθώς ανταλλάζει την κατασκευή πάρα πολλών λιστών με ελέγχους ροής που δεν γίνονται (θα μπορούσαν να δημιουργούνταν μόνο όσες λίστες χρειάζεται). Μία άλλη πιθανή πηγή της επιτάχυνσης είναι το μικρότερο κόστος memory management, allocation/deallocaiton.